

# UE Apprentissage SIA S9

Supervise Learning

Mathieu FAUVEL

CESBIO, Université de Toulouse, CNES/CNRS/INRAe/IRD/UPS, Toulouse, FRANCE

September 25, 2023

# Outline

Introduction

Introductory Example

Multi layer Perceptron

Convolutional Neural Networks

# Outline

## Introduction

### Introductory Example

- Linear model for regression

- Linear model for classification

### Multi layer Perceptron

- Going deeper

- Needs some regularization

### Convolutional Neural Networks

- Concepts

- How to improve the learning process

- Contact: [mathieu.fauvel@inrae.fr](mailto:mathieu.fauvel@inrae.fr)
- CV:

2004-2007 Ph.D. degree in Signal and Image Processing from the INP, Grenoble & the University of Iceland

2007-2008 Assistant Professor Grenoble

2008-2010 Post-doc position at INRIA - MISTIS Team

2010-2011 Assistant Professor Toulouse

2011-2018 Associate Professor at DYNAFOR & INP, Toulouse

Since 2018 Research (CRCN) at CESBIO, INRAe

- Research interests are:

Machine learning for environmental/ecological monitoring

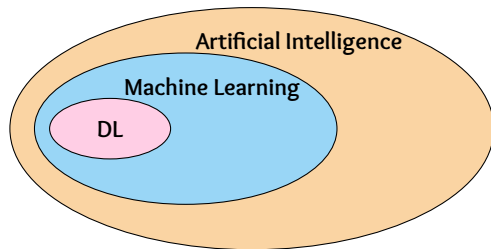
# Course Objectives

- Learn basics of modern machine learning
- Understand how each step works
  - ★ Data préparation
  - ★ Model definition
  - ★ Optimization step
- Implement various Deep Learning models in PyTorch
- Application to Computer Vision

# What is Supervised (Machine) Learning ?

- **Artificial Intelligence**

*Perform human tasks using computers and algorithms*



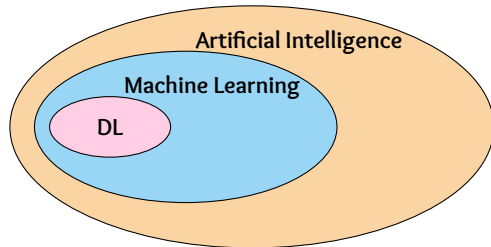
# What is Supervised (Machine) Learning ?

- **Artificial Intelligence**

*Perform human tasks using computers and algorithms*

- Mitchell, *The discipline of machine learning:*

*Machine Learning is defined as the capacity of a computer program to improve its performance measure with observations*



# What is Supervised (Machine) Learning ?

- **Artificial Intelligence**

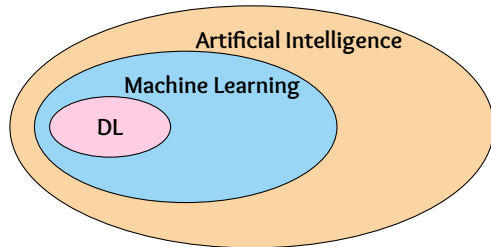
*Perform human tasks using computers and algorithms*

- Mitchell, *The discipline of machine learning:*

*Machine Learning is defined as the capacity of a computer program to improve its performance measure with observations*

- **Deep Learning**

- ★ Lot of data data
- ★ Complex model
- ★ High performance computing



# Main Notations

## Data

- Observed data  $\mathbf{x} \in \mathbb{R}^d$ , called *input variables* or *predictors*.
- Data to be predicted  $y \in \mathbb{R}^p$ , called *output variables* or *responses*.

# Main Notations

## Data

- Observed data  $\mathbf{x} \in \mathbb{R}^d$ , called *input variables* or *predictors*.
- Data to be predicted  $y \in \mathbb{R}^p$ , called *output variables* or *responses*.

## Learning Problem

- If  $y$  are quantitative data: *regression* and  $y \in \mathbb{R}^p$ .
- If  $y$  are categorical data: *classification* and  $y \in \{C_1, \dots, C_C\}$  with  $C_i$  refers to the  $i^{\text{th}}$  category.

# Main Notations

## Data

- Observed data  $\mathbf{x} \in \mathbb{R}^d$ , called *input variables* or *predictors*.
- Data to be predicted  $y \in \mathbb{R}^p$ , called *output variables* or *responses*.

## Learning Problem

- If  $y$  are quantitative data: *regression* and  $y \in \mathbb{R}^p$ .
- If  $y$  are categorical data: *classification* and  $y \in \{C_1, \dots, C_C\}$  with  $C_i$  refers to the  $i^{\text{th}}$  category.

## Prediction function

$$\begin{aligned} f_{\theta} : \mathbb{R}^d &\rightarrow \mathbb{R}^p \\ \mathbf{x} &\mapsto y \end{aligned}$$

# Online References

- Aston Zhang et al. “Dive into Deep Learning”. In: *arXiv preprint arXiv:2106.11342* (2021)
- Simon J.D. Prince. *Understanding Deep Learning*. MIT Press, 2023. URL: <https://udlbook.github.io/udlbook/>
- Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL: <http://probml.github.io/book1>
- Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL: <http://probml.github.io/book2>

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

- Needs some regularization

## Convolutional Neural Networks

- Concepts

- How to improve the learning process

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

- Needs some regularization

## Convolutional Neural Networks

- Concepts

- How to improve the learning process

# Univariate linear model

- $f(x) = wx + b$  and  $\theta = (w, b)$
- Loss function:  $\ell(f(x_i), y_i) = (f(x_i) - y_i)^2$
- Objective function:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell_i$$

- Gradients update:  $\theta^{t+1} = \theta^t - \eta \nabla_{\theta} \mathcal{L}$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{n} \sum_{i=1}^n x_i (wx_i + b - y_i)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n} \sum_{i=1}^n (wx_i + b - y_i)$$

# Univariate linear model

- $f(x) = wx + b$  and  $\theta = (w, b)$
- Loss function:  $\ell(f(x_i), y_i) = (f(x_i) - y_i)^2$
- Objective function:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell_i$$

- Gradients update:  $\theta^{t+1} = \theta^t - \eta \nabla_{\theta} \mathcal{L}$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{n} \sum_{i=1}^n x_i (wx_i + b - y_i)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n} \sum_{i=1}^n (wx_i + b - y_i)$$

## Work

Implement the 1D regression in pytorch of the following function:

$$f(x) = 2x - 1$$

Do the notebook:

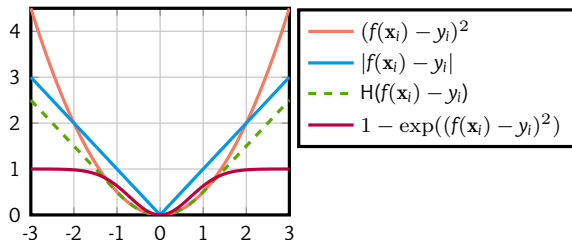
[regression\\_1d\\_toy.ipynb](#)

# Multivariate linear model

- $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$  and  $\boldsymbol{\theta} = (\mathbf{w}, b)$
- Same loss and objective function and so same gradient updates ...

# Multivariate linear model

- $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$  and  $\boldsymbol{\theta} = (\mathbf{w}, b)$
- Same loss and objective function and so same gradient updates ...
- We can use other loss function

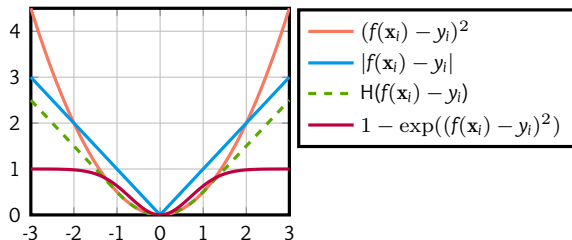


with H stands for *Hubert* loss:

$$H(f(\mathbf{x}_i) - y_i) = \begin{cases} \frac{1}{2} (f(\mathbf{x}_i) - y_i)^2 & \text{for } |f(\mathbf{x}_i) - y_i| \leq \delta, \\ (|f(\mathbf{x}_i) - y_i| - 0.5), & \text{otherwise.} \end{cases}$$

# Multivariate linear model

- $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$  and  $\boldsymbol{\theta} = (\mathbf{w}, b)$
- Same loss and objective function and so same gradient updates ...
- We can use other loss function



with H stands for *Hubert loss*:

$$H(f(\mathbf{x}_i) - y_i) = \begin{cases} \frac{1}{2} (f(\mathbf{x}_i) - y_i)^2 & \text{for } |f(\mathbf{x}_i) - y_i| \leq \delta, \\ (|f(\mathbf{x}_i) - y_i| - 0.5), & \text{otherwise.} \end{cases}$$

## Work

On the **California housing** data set

- Prepare (train/validation) data
- Define a multi. linear model
- Optimize your hyperparameters (batch size, learning rate)
- Switch L2 to other loss function

# Scaling feature for multivariate linear model with L2 loss function

- Suppose we have two features of different scale (e.g. because of different unit)

$$\mathbf{x}_1 \sim \mathcal{N}(0, 1) \text{ and } \mathbf{x}_2 \sim \mathcal{N}(10, 10)$$

- What is the impact on the gradient update ?

# Scaling feature for multivariate linear model with L2 loss function

- Suppose we have two features of different scale (e.g. because of different unit)

$$\mathbf{x}_1 \sim \mathcal{N}(0, 1) \text{ and } \mathbf{x}_2 \sim \mathcal{N}(10, 10)$$

- What is the impact on the gradient update ?
- Noting  $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top$  and  $\mathbf{y} = [y_1, \dots, y_n]^\top$

$$\nabla_{\mathbf{w}} = -\mathbf{X}^\top \underbrace{(\mathbf{y} - \mathbf{X}\mathbf{w})}_{\mathbf{e}} \Rightarrow \nabla_{\mathbf{w}_p} = -\sum_{i=1}^n \mathbf{e}_i \mathbf{x}_{ip}$$

- Parameter update

$$\mathbf{w}_p^{(t+1)} = \mathbf{w}_p^{(t)} + \frac{\eta}{n} \sum_{i=1}^n \mathbf{e}_i^{(t)} \mathbf{x}_{ip}$$

# Scaling feature for multivariate linear model with L2 loss function

- Suppose we have two features of different scale (e.g. because of different unit)

$$\mathbf{x}_1 \sim \mathcal{N}(0, 1) \text{ and } \mathbf{x}_2 \sim \mathcal{N}(10, 10)$$

- What is the impact on the gradient update ?
- Noting  $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top$  and  $\mathbf{y} = [y_1, \dots, y_n]^\top$

$$\nabla_{\mathbf{w}} = -\mathbf{X}^\top \underbrace{(\mathbf{y} - \mathbf{X}\mathbf{w})}_{\mathbf{e}} \Rightarrow \nabla_{\mathbf{w}_p} = -\sum_{i=1}^n \mathbf{e}_i \mathbf{x}_{ip}$$

- Parameter update

$$\mathbf{w}_p^{(t+1)} = \mathbf{w}_p^{(t)} + \frac{\eta}{n} \sum_{i=1}^n \mathbf{e}_i^{(t)} \mathbf{x}_{ip}$$

- Standardization of the input feature (and for regression the output too): [scaling\\_feature.ipynb](#)

$$\tilde{\mathbf{x}} = (\mathbf{x} - \boldsymbol{\mu}) \oslash \boldsymbol{\sigma}$$

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

- Needs some regularization

## Convolutional Neural Networks

- Concepts

- How to improve the learning process

# Classification as a regression problem

- Predict categorical variables ("Dogs", "Cats", "Monkey" ...)
- Estimation of the posterior probability

$$p(y_i = c | \mathbf{x}_i) \forall c \in \{1, \dots, C\} \text{ with } p(y_i = c | \mathbf{x}_i) \geq 0 \text{ and } \sum_{c=1}^C p(y_i = c | \mathbf{x}_i) = 1$$

- MAP rule:  $\hat{c}_i = \arg \max_c p(y_i = c | \mathbf{x}_i)$

# Classification as a regression problem

- Predict categorical variables ("Dogs", "Cats", "Monkey" ...)
- Estimation of the posterior probability

$$p(y_i = c | \mathbf{x}_i) \forall c \in \{1, \dots, C\} \text{ with } p(y_i = c | \mathbf{x}_i) \geq 0 \text{ and } \sum_{c=1}^C p(y_i = c | \mathbf{x}_i) = 1$$

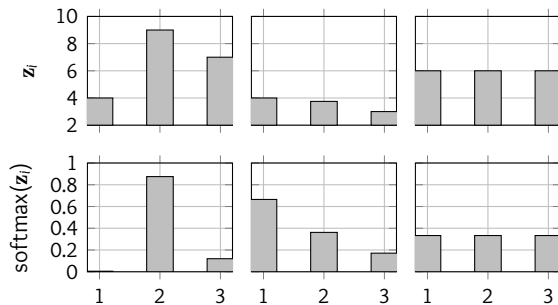
- MAP rule:  $\hat{c}_i = \arg \max_c p(y_i = c | \mathbf{x}_i)$
- Softmax linear regression

$$1 = \sum_{c=1}^C p(y_i = c | \mathbf{x}_i) = \sum_{c=1}^C \exp \left\{ \mathbf{w}_c^\top \mathbf{x}_i + b_c + K_i \right\} = \exp \{K_i\} \sum_{c=1}^C \exp \left\{ \mathbf{w}_c^\top \mathbf{x}_i + b_c \right\}$$

thus

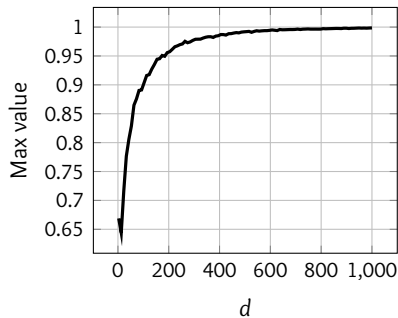
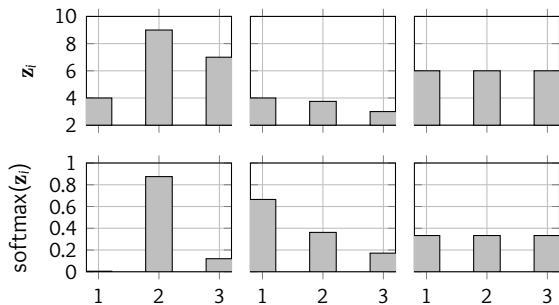
$$p(y_i = c | \mathbf{x}_i) = \frac{\exp \left\{ \mathbf{w}_c^\top \mathbf{x}_i + b_c \right\}}{\sum_{c'=1}^C \exp \left\{ \mathbf{w}_{c'}^\top \mathbf{x}_i + b_{c'} \right\}} = \frac{\exp(\mathbf{z}_{ic})}{\sum_{c'=1}^C \exp(\mathbf{z}_{ic'})} = \text{softmax}(\mathbf{z}_i)_c$$

# Softmax



- Translation invariant:  $\text{softmax}(\mathbf{z} + \mathbf{K}) = \text{softmax}(\mathbf{z})$
- Preserve ordering relation:  $\arg \max_c(\mathbf{p}_i) = \arg \max_c(\mathbf{z}_i)$
- If  $\mathbf{z} \in \mathbb{R}^d$  with  $d \rightarrow \infty$  then  $\text{softmax}(\mathbf{z}) \rightarrow \text{"One-hot-vector"}$

# Softmax



- Translation invariant:  $\text{softmax}(\mathbf{z} + K) = \text{softmax}(\mathbf{z})$
- Preserve ordering relation:  $\arg \max_c(\mathbf{p}_i) = \arg \max_c(\mathbf{z}_i)$
- If  $\mathbf{z} \in \mathbb{R}^d$  with  $d \rightarrow \infty$  then  $\text{softmax}(\mathbf{z}) \rightarrow \text{"One-hot-vector"}$

# Cross entropy loss function

- One hot encoding:  $y_i = c \rightarrow \mathbf{y}_i \in \mathbb{R}^C$ ,  $\mathbf{y}_{ic} = 1$  if  $i = c$  else 0
- Likelihood:

$$l(\mathbf{y}_i, \hat{\mathbf{y}}_i) = \prod_{c'=1}^C p(y_i = c' | \mathbf{x}_i)^{\mathbf{y}_{ic'}} = \frac{\exp(\mathbf{z}_{ic})}{\sum_{c'=1}^C \exp(\mathbf{z}_{ic'})}$$

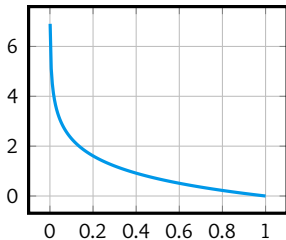
# Cross entropy loss function

- One hot encoding:  $y_i = c \rightarrow \mathbf{y}_i \in \mathbb{R}^C$ ,  $y_{ic} = 1$  if  $i = c$  else 0
- Likelihood:

$$l(\mathbf{y}_i, \hat{\mathbf{y}}_i) = \prod_{c'=1}^C p(y_i = c' | \mathbf{x}_i)^{y_{ic'}} = \frac{\exp(\mathbf{z}_{ic})}{\sum_{c'=1}^C \exp(\mathbf{z}_{ic'})}$$

- Negative log likelihood:

$$\begin{aligned}\ell(\mathbf{y}_i, \hat{\mathbf{y}}_i) &= - \sum_{c'=1}^C y_{ic'} \log \{p(y_i = c' | \mathbf{x}_i)\} \\ &= -\log \{p(y_i = c | \mathbf{x}_i)\} \\ &= -(\mathbf{w}_c^\top \mathbf{x}_i + b_c) + \underbrace{\log \left\{ \sum_{c'=1}^C \exp(\mathbf{w}_{c'}^\top \mathbf{x}_i + b_{c'}) \right\}}_{\text{logsumexp}(\mathbf{z}_i)}\end{aligned}$$



## Work

What happens if:

- All  $\mathbf{z}_{ic}$  are small ?
- One  $\mathbf{z}_{ic}$  is very large ?
- All  $\mathbf{z}_{ic}$  are very large ?

# Multivariate linear classifier

- The model

$$\mathbf{p}_i = \text{softmax}(\mathbf{W}\mathbf{x}_i + \mathbf{b})$$

- Cross entropy loss function
- Same comments than for regression
  - ★ Split train, val and test
  - ★ Standardization
  - ★ Batch training
  - ★ Tune learning rate

## Work

Implement a linear classifier in pytorch for the classification of **Fashion MNIST** data set.

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

- Needs some regularization

## Convolutional Neural Networks

- Concepts

- How to improve the learning process

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

- Needs some regularization

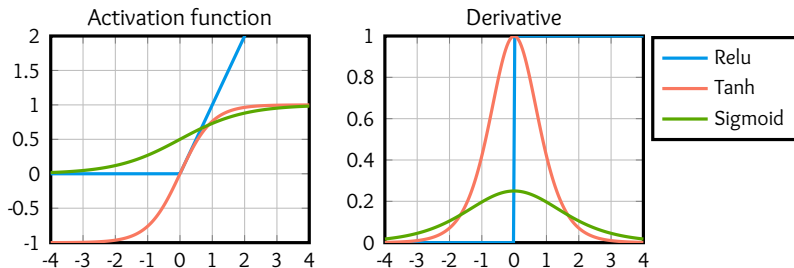
## Convolutional Neural Networks

- Concepts

- How to improve the learning process

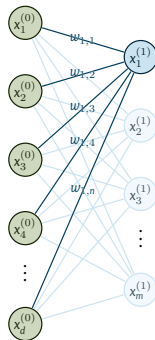
# Activation function

- As discussed earlier we need non-linearity between two layers
- Done with *activation function* between two layers
  - ★ Rectified Linear (ReLU):  $\max(x, 0)$
  - ★ Sigmoid:  $\frac{1}{1 + \exp(-x)}$
  - ★ Tanh:  $\tanh(x)$



# Multi Layer Perceptron (MLP)

- $\mathbf{x}^{(1)} = \sigma(\mathbf{W}^{(0)}\mathbf{x}^{(0)} + \mathbf{b}^{(0)})$
- $\mathbf{x}^{(2)} = \sigma(\mathbf{W}^{(1)}\mathbf{x}^{(1)} + \mathbf{b}^{(1)})$
- ...
- $\mathbf{y} = \mathbf{W}^H\mathbf{x}^H + \mathbf{b}^H$



$$\begin{aligned} &= \sigma(w_{1,0}x_0^{(0)} + w_{1,1}x_1^{(0)} + \dots + w_{1,d}x_d^{(0)} + b_1^{(0)}) \\ &= \sigma\left(\sum_{i=1}^d w_{1,i}x_i^{(0)} + b_1^{(0)}\right) \end{aligned}$$

$$\begin{pmatrix} x_1^{(1)} \\ x_2^{(1)} \\ \vdots \\ x_m^{(1)} \end{pmatrix} = \sigma \left[ \begin{pmatrix} w_{1,0} & w_{1,1} & \dots & w_{1,d} \\ w_{2,0} & w_{2,1} & \dots & w_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,0} & w_{m,1} & \dots & w_{m,d} \end{pmatrix} \begin{pmatrix} x_1^{(0)} \\ x_2^{(0)} \\ \vdots \\ x_d^{(0)} \end{pmatrix} + \begin{pmatrix} b_1^{(0)} \\ b_2^{(0)} \\ \vdots \\ b_m^{(0)} \end{pmatrix} \right]$$
$$\mathbf{x}^{(1)} = \sigma(\mathbf{W}^{(0)}\mathbf{x}^{(0)} + \mathbf{b}^{(0)})$$

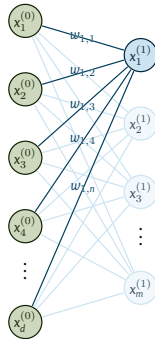
From [https://tikz.net/neural\\_networks/](https://tikz.net/neural_networks/)

# Multi Layer Perceptron (MLP)

- $\mathbf{x}^{(1)} = \sigma(\mathbf{W}^{(0)}\mathbf{x}^{(0)} + \mathbf{b}^{(0)})$
- $\mathbf{x}^{(2)} = \sigma(\mathbf{W}^{(1)}\mathbf{x}^{(1)} + \mathbf{b}^{(1)})$
- ...
- $\mathbf{y} = \mathbf{W}^H\mathbf{x}^H + \mathbf{b}^H$

## Work

- Why the last layer does not have non-linearity?
- For an univariate regression problem, shows that a 2-layers MLP is a piece-wise linear function.
- Implement an MLP classifier for Fashion MNIST data set



$$= \sigma(w_{1,0}x_0^{(0)} + w_{1,1}x_1^{(0)} + \dots + w_{1,d}x_d^{(0)} + b_1^{(0)})$$

$$= \sigma\left(\sum_{i=1}^d w_{1,i}x_i^{(0)} + b_1^{(0)}\right)$$

$$\begin{pmatrix} x_1^{(1)} \\ x_2^{(1)} \\ \vdots \\ x_m^{(1)} \end{pmatrix} = \sigma \left[ \begin{pmatrix} w_{1,0} & w_{1,1} & \dots & w_{1,d} \\ w_{2,0} & w_{2,1} & \dots & w_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,0} & w_{m,1} & \dots & w_{m,d} \end{pmatrix} \begin{pmatrix} x_1^{(0)} \\ x_2^{(0)} \\ \vdots \\ x_d^{(0)} \end{pmatrix} + \begin{pmatrix} b_1^{(0)} \\ b_2^{(0)} \\ \vdots \\ b_m^{(0)} \end{pmatrix} \right]$$

$$\mathbf{x}^{(1)} = \sigma(\mathbf{W}^{(0)}\mathbf{x}^{(0)} + \mathbf{b}^{(0)})$$

From [https://tikz.net/neural\\_networks/](https://tikz.net/neural_networks/)

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

- Needs some regularization

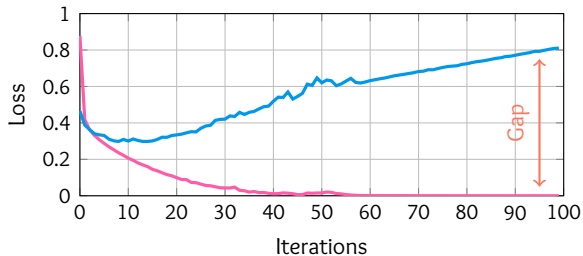
## Convolutional Neural Networks

- Concepts

- How to improve the learning process

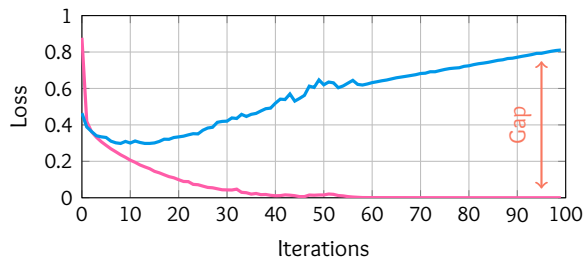
# Generalization Error

- *Generalization error*: Difference between the train accuracy and val/test accuracy
- *Overfitting*: High generalization error



# Generalization Error

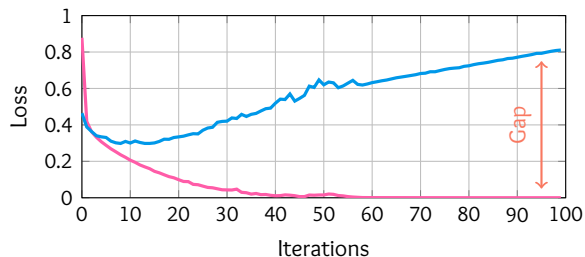
- *Generalization error*: Difference between the train accuracy and val/test accuracy
- *Overfitting*: High generalization error



**Theory:** favor simpler model and/or smooth model

# Generalization Error

- *Generalization error*: Difference between the train accuracy and val/test accuracy
- *Overfitting*: High generalization error

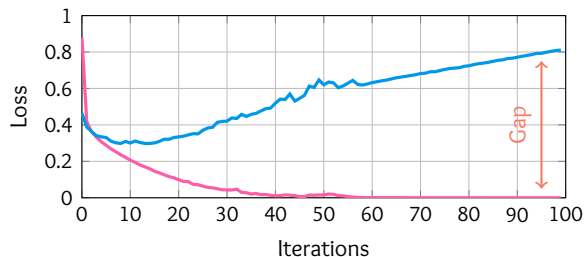


**Theory:** favor simpler model and/or smooth model

- Early stopping: monitor the validation loss

# Generalization Error

- *Generalization error*: Difference between the train accuracy and val/test accuracy
- *Overfitting*: High generalization error



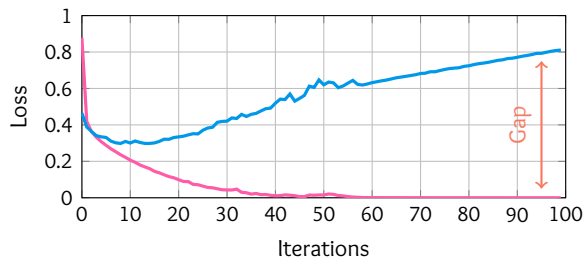
**Theory:** favor simpler model and/or smooth model

- Early stopping: monitor the validation loss
- Weight decay: Tikhonov regularization

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell_i + \frac{\lambda}{H} \sum_{h=1}^H \|\mathbf{w}_h\|_2^2$$

# Generalization Error

- *Generalization error*: Difference between the train accuracy and val/test accuracy
- *Overfitting*: High generalization error



**Theory:** favor simpler model and/or smooth model

- Early stopping: monitor the validation loss
- Weight decay: Tikhonov regularization
- Noise injection: Dropout

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell_i + \frac{\lambda}{H} \sum_{h=1}^H \|\mathbf{w}_h\|_2^2$$

# Dropout

- Smoothness:  $f(\mathbf{x} + \epsilon) \approx f(\mathbf{x})$
- Idea: Noise injection during the learning process such as  $\mathbb{E}[\tilde{\mathbf{x}}] = \mathbf{x}$
- With MLP:

$$f(\mathbf{x}) = f_H \circ f_{H-1} \circ \dots \circ f_0(\mathbf{x})$$

$$\text{and } \mathbf{x}^{(h+1)} = f_h(\mathbf{x}^{(h)}) = \sigma(\mathbf{W}^{(h)}\mathbf{x}^{(h)} + \mathbf{b}^{(h)})$$

- Dropout: remove (set to zero) some internal features with probability  $p$

$$\mathbf{x}^{(h+1)} = f_h(\tilde{\mathbf{x}}^{(h)}) = \sigma(\mathbf{W}^{(h)}(\mathbf{m}^{(h)} \odot \mathbf{x}^{(h)}) + \mathbf{b}^{(h)})$$

with

$$\mathbf{m}_i^{(h)} = \begin{cases} 0 & \text{with probability } p \\ 1/(1-p) & \text{with probability } 1-p \end{cases}$$

# Dropout

- Smoothness:  $f(\mathbf{x} + \epsilon) \approx f(\mathbf{x})$
- Idea: Noise injection during the learning process such as  $\mathbb{E}[\tilde{\mathbf{x}}] = \mathbf{x}$
- With MLP:

$$f(\mathbf{x}) = f_H \circ f_{H-1} \circ \dots \circ f_0(\mathbf{x})$$

$$\text{and } \mathbf{x}^{(h+1)} = f_h(\mathbf{x}^{(h)}) = \sigma(\mathbf{W}^{(h)}\mathbf{x}^{(h)} + \mathbf{b}^{(h)})$$

- Dropout: remove (set to zero) some internal features with probability  $p$

$$\mathbf{x}^{(h+1)} = f_h(\tilde{\mathbf{x}}^{(h)}) = \sigma(\mathbf{W}^{(h)}(\mathbf{m}^{(h)} \odot \mathbf{x}^{(h)}) + \mathbf{b}^{(h)})$$

with

$$\mathbf{m}_i^{(h)} = \begin{cases} 0 & \text{with probability } p \\ 1/(1-p) & \text{with probability } 1-p \end{cases}$$

- Enable during training and disable during inference (can be used to estimate posterior distribution)

$$f(\tilde{\mathbf{x}}) = f_H \circ D_H \circ f_{H-1} \circ \dots \circ D_1 \circ f_0(\mathbf{x})$$

## Work

- Implement early stopping
- Implement dropout
- (Optional) Implement Tikhonov regularization (weight decay in pytorch)

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

- Needs some regularization

## Convolutional Neural Networks

- Concepts

- How to improve the learning process

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

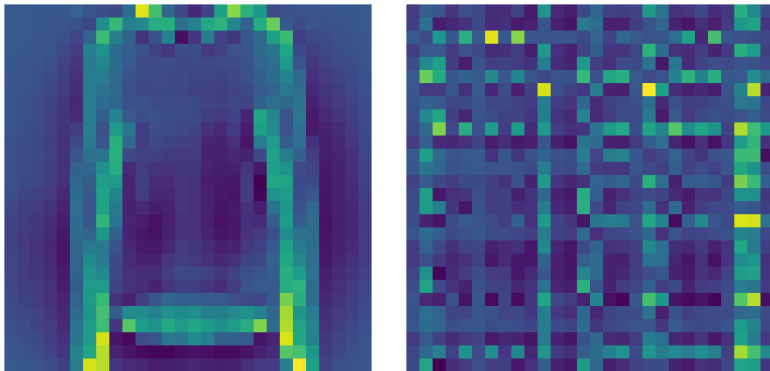
- Needs some regularization

## Convolutional Neural Networks

- Concepts

- How to improve the learning process

# An image is not a vector of pixels



# 2D Convolution

- Fully connected layer:

$$x_{pq}^{(1)} = \sigma \left( \sum_{i,j=1}^{H,W} \mathbf{w}_{pqij}^{(0)} x_{ij}^{(0)} + b_{pq}^{(0)} \right)$$

- Act locally: restrict to neighborhood of pixel  $p, q$

$$x_{pq}^{(1)} = \sigma \left( \sum_{i,j=-\Delta}^{i,j=+\Delta} \mathbf{w}_{ij}^{(0)} x_{(p+i)(j+q)}^{(0)} + b^{(0)} \right)$$

$$\begin{pmatrix} w_{pq11} & w_{pq12} & \dots & w_{pq1j} & \dots & w_{pq1W} \\ w_{pq21} & w_{pq22} & \dots & w_{pq2j} & \dots & w_{pq2W} \\ \vdots & \vdots & & \vdots & & \vdots \\ w_{pqi1} & w_{pqi2} & \dots & w_{pqij} & \dots & w_{pqiW} \\ \vdots & \vdots & & \vdots & & \vdots \\ w_{pqH1} & w_{pqH2} & \dots & w_{pqHj} & \dots & w_{pqHW} \end{pmatrix}$$

# 2D Convolution

- Fully connected layer:

$$x_{pq}^{(1)} = \sigma \left( \sum_{i,j=1}^{H,W} \mathbf{w}_{pqij}^{(0)} x_{ij}^{(0)} + b_{pq}^{(0)} \right)$$

- Act locally: restrict to neighborhood of pixel  $p, q$

$$x_{pq}^{(1)} = \sigma \left( \sum_{i,j=-\Delta}^{i,j=+\Delta} \mathbf{w}_{ij}^{(0)} x_{(p+i)(j+q)}^{(0)} + b^{(0)} \right)$$

$$\begin{pmatrix} 0 & \dots & 0 & \dots & 0 \\ 0 & \dots & 0 & \dots & 0 \\ \vdots & w_{(i)(j-1)} & w_{(i-1)(j)} & w_{(i-1)(j+1)} & \vdots \\ 0 & w_{(i)(j-1)} & w_{(i)(j)} & w_{(i)(j+1)} & 0 \\ \vdots & w_{(i+1)(j-1)} & w_{(i+1)(j)} & w_{(i+1)(j+1)} & \vdots \\ 0 & \dots & 0 & \dots & 0 \end{pmatrix}$$

# 2D Convolution

- Fully connected layer:

$$x_{pq}^{(1)} = \sigma \left( \sum_{i,j=1}^{H,W} \mathbf{w}_{pqij}^{(0)} x_{ij}^{(0)} + b_{pq}^{(0)} \right)$$

- Act locally: restrict to neighborhood of pixel  $p, q$

$$x_{pq}^{(1)} = \sigma \left( \sum_{i,j=-\Delta}^{i,j=+\Delta} \mathbf{w}_{ij}^{(0)} x_{(p+i)(q+j)}^{(0)} + b^{(0)} \right)$$

$$\begin{pmatrix} 0 & \dots & 0 & \dots & 0 \\ 0 & \dots & 0 & \dots & 0 \\ \vdots & w_{(i)(j-1)} & w_{(i-1)(j)} & w_{(i-1)(j+1)} & \vdots \\ 0 & w_{(i)(j-1)} & w_{(i)(j)} & w_{(i)(j+1)} & 0 \\ \vdots & w_{(i+1)(j-1)} & w_{(i+1)(j)} & w_{(i+1)(j+1)} & \vdots \\ 0 & \dots & 0 & \dots & 0 \end{pmatrix}$$

# 2D Convolution

- Fully connected layer:

$$x_{pq}^{(1)} = \sigma \left( \sum_{i,j=1}^{H,W} \mathbf{w}_{pqij}^{(0)} x_{ij}^{(0)} + b_{pq}^{(0)} \right)$$

- Act locally: restrict to neighborhood of pixel  $p, q$

$$x_{pq}^{(1)} = \sigma \left( \sum_{i,j=-\Delta}^{i,j=+\Delta} \mathbf{w}_{ij}^{(0)} x_{(p+i)(j+q)}^{(0)} + b^{(0)} \right)$$

$$\begin{pmatrix} 0 & \dots & 0 & \dots & 0 \\ 0 & \dots & 0 & \dots & 0 \\ \vdots & w_{(i)(j-1)} & w_{(i-1)(j)} & w_{(i-1)(j+1)} & \vdots \\ 0 & w_{(i)(j-1)} & w_{(i)(j)} & w_{(i)(j+1)} & 0 \\ \vdots & w_{(i+1)(j-1)} & w_{(i+1)(j)} & w_{(i+1)(j+1)} & \vdots \\ 0 & \dots & 0 & \dots & 0 \end{pmatrix}$$

## Work

Show that 2D convolution can be expressed as a linear layer.

# Basics on 2D Convolution 1/2

## Convolution

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 |

$\chi^{(h)}$

\*

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

$K$

=

|    |     |     |
|----|-----|-----|
| 54 | 63  | 72  |
| 99 | 108 | 117 |

$\chi^{(h+1)}$

(More [https://github.com/vdumoulin/conv\\_arithmetic/blob/master/README.md](https://github.com/vdumoulin/conv_arithmetic/blob/master/README.md))

# Basics on 2D Convolution 1/2

## Convolution + Padding (1,1)

|   |    |    |    |    |    |   |
|---|----|----|----|----|----|---|
| 0 | 0  | 0  | 0  | 0  | 0  | 0 |
| 0 | 0  | 1  | 2  | 3  | 4  | 0 |
| 0 | 5  | 6  | 7  | 8  | 9  | 0 |
| 0 | 10 | 11 | 12 | 13 | 14 | 0 |
| 0 | 15 | 16 | 17 | 18 | 19 | 0 |
| 0 | 0  | 0  | 0  | 0  | 0  | 0 |

\*

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

=

|    |    |     |     |    |
|----|----|-----|-----|----|
| 12 | 21 | 27  | 33  | 24 |
| 33 | 54 | 63  | 72  | 51 |
| 63 | 99 | 108 | 117 | 81 |
| 52 | 81 | 87  | 93  | 64 |

# Basics on 2D Convolution 1/2

Convolution + Padding (1,1) + Stride (3, 2)

|   |    |    |    |    |    |   |
|---|----|----|----|----|----|---|
| 0 | 0  | 0  | 0  | 0  | 0  | 0 |
| 0 | 0  | 1  | 2  | 3  | 4  | 0 |
| 0 | 5  | 6  | 7  | 8  | 9  | 0 |
| 0 | 10 | 11 | 12 | 13 | 14 | 0 |
| 0 | 15 | 16 | 17 | 18 | 19 | 0 |
| 0 | 0  | 0  | 0  | 0  | 0  | 0 |

\*

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

=

|    |    |    |
|----|----|----|
| 12 | 27 | 24 |
| 52 | 87 | 64 |

# Basics on 2D Convolution 1/2

## Convolution for multivalued images

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 |

\*

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

=

|    |     |     |
|----|-----|-----|
| 54 | 63  | 72  |
| 99 | 108 | 117 |

+

=

|     |     |     |
|-----|-----|-----|
| 288 | 279 | 270 |
| 243 | 234 | 225 |

|    |    |    |    |    |
|----|----|----|----|----|
| 19 | 18 | 17 | 16 | 15 |
| 14 | 13 | 12 | 11 | 10 |
| 9  | 8  | 7  | 6  | 5  |
| 4  | 3  | 2  | 1  | 0  |

\*

|   |   |   |
|---|---|---|
| 2 | 2 | 2 |
| 2 | 2 | 2 |
| 2 | 2 | 2 |

=

|     |     |     |
|-----|-----|-----|
| 234 | 216 | 198 |
| 144 | 126 | 108 |

# Basics of 2D Convolution 2/2

- Multivariate input - Multivariate output

$$\mathbf{X}^{(h)} \in \mathbb{R}^{C_{(h)} \times H^{(h)} \times W^{(h)}}$$

$$\mathbf{K}^{(h)} \in \mathbb{R}^{C_{(h+1)} \times C_{(h)} \times k_H^h \times k_W^h}$$

$$\mathbf{X}^{(h+1)} \in \mathbb{R}^{C_{(h+1)} \times H^{(h+1)} \times W^{(h+1)}}$$

- Special case of  $1 \times 1$  convolution:  $(C_{(h+1)}, C_{(h)}, k_H, k_W) = (C_o, C_i, 1, 1)$ 
  - ★ Combine pixel across channels - no spatial operation
  - ★ Matrix product / linear layer in the channel dimension

$$\mathbf{X}^{(h+1)} = \text{reshape} \left\{ \sigma \left( \mathbf{K}^{(h)} \text{reshape} \{ \mathbf{X}^{(h)}, C_{(h)}, H^{(h)} W^{(h)} \} + \mathbf{b}^{(h)} \right), C_{(h+1)}, H^{(h)}, W^{(h)} \right\}$$

with  $\text{reshape} \{ \mathbf{X}^{(h)}, C_{(h)}, H^{(h)} W^{(h)} \} \in \mathbb{R}^{C_{(h)} \times H^{(h)} W^{(h)}}$

# Pooling

- Reduce the resolution of the image  $\equiv$  Use larger convolution kernel
- Downsampling with simple operator on the neighborhood:
  - ★ Average value
  - ★ Max value (preferred)
- After the non-linearity

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 |

$2 \times 2$  MaxPooling  
Stride=2

|    |    |
|----|----|
| 6  | 8  |
| 16 | 18 |

## Work

- Express multi-channels & multi-outputs convolution as a matrix product.
- Show that the successive convolution with two kernels is equivalent to one convolution.
- What stride size will you use for a max pooling with a kernel of size  $2 \times 2$ ?
- Implement a CNN for the classification of *Fashion MNIST* data set: use one or two convolutional layers for the MLP.

# Outline

## Introduction

## Introductory Example

- Linear model for regression

- Linear model for classification

## Multi layer Perceptron

- Going deeper

- Needs some regularization

## Convolutional Neural Networks

- Concepts

- How to improve the learning process

# Batch Normalization

- Remember scaling problem in slide 10
- Similar problem may happen to each layer **but** we cannot use the same trick: the sample mean/variance change after each batch update !

# Batch Normalization

- Remember scaling problem in slide 10
- Similar problem may happen to each layer **but** we cannot use the same trick: the sample mean/variance change after each batch update !
- Apply normalization per batch: *Batch Normalization*

$$\tilde{\mathbf{x}}_B = \gamma \odot \left[ (\mathbf{x}_B - \boldsymbol{\mu}_B) \oslash \boldsymbol{\sigma}_B \right] + \boldsymbol{\beta}$$

with  $\boldsymbol{\mu}_B$  and  $\boldsymbol{\sigma}_B$  computed on batch  $B$ , and  $\gamma$  &  $\boldsymbol{\beta}$  are learnable parameters

# Batch Normalization

- Remember scaling problem in slide 10
- Similar problem may happen to each layer **but** we cannot use the same trick: the sample mean/variance change after each batch update !
- Apply normalization per batch: *Batch Normalization*

$$\tilde{\mathbf{x}}_B = \gamma \odot \left[ (\mathbf{x}_B - \boldsymbol{\mu}_B) \oslash \boldsymbol{\sigma}_B \right] + \boldsymbol{\beta}$$

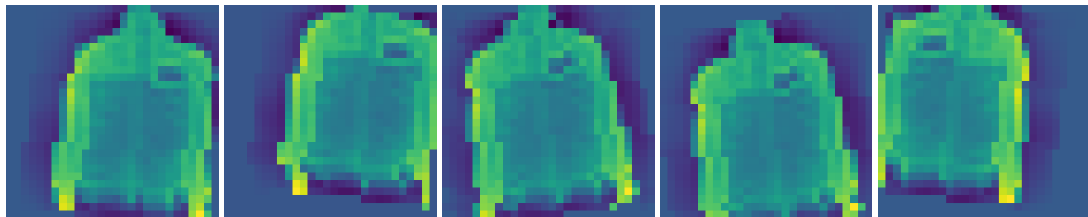
with  $\boldsymbol{\mu}_B$  and  $\boldsymbol{\sigma}_B$  computed on batch  $B$ , and  $\gamma$  &  $\boldsymbol{\beta}$  are learnable parameters

## Work

- What happen if the batch size is one ?
- What happen if the batch size is small ?
- What should we do in *prediction* mode ?
- Add normalization layer (2D) to your model.

# Data Augmentation

- Noise injection to generate / augment data set:  $(\mathbf{x}, y) \rightarrow (\tilde{\mathbf{x}}, y)$
- For image classification it is *relatively* easy:
  - ★ Geometric transformation: Rotation, translation, symmetry, radiometry ...
  - ★ Radiometric transformation: Invert, jitter ...
  - ★ *Any transformation that can model the variability of the acquisition process*



## Work

Implement data augmentation with Pytorch for the training data only.

- Deep Neural Network:  $f_h(\mathbf{x}^{(h)}) := f_h(\mathbf{x}^{(h)}, \theta_h)$

$$f(\mathbf{x}) = f_H \circ f_{H-1} \circ \dots \circ f_0(\mathbf{x})$$

# Residual Network

- Deep Neural Network:  $f_h(\mathbf{x}^{(h)}) := f_h(\mathbf{x}^{(h)}, \theta_h)$

$$f(\mathbf{x}) = f_H \circ f_{H-1} \circ \dots \circ f_0(\mathbf{x})$$

- Let  $f_h = g_2 \circ g_1 \circ g_0$

$$\mathbf{x} \xrightarrow{g_0} \mathbf{x}^{(1)} \xrightarrow{g_1} \mathbf{x}^{(2)} \xrightarrow{g_2} f_h(\mathbf{x})$$

$$\frac{\partial f_h(\mathbf{x})}{\partial \alpha_0} = \underbrace{\frac{g_2}{g_1} \circ \frac{g_1}{g_0}}_{\rightarrow} \frac{g_0}{\alpha_0}(\mathbf{x}) \quad \frac{\partial g(\mathbf{x})}{\partial \alpha_1} = \frac{g_2}{g_1} \circ \frac{g_1}{\alpha_1}(\mathbf{x}^{(1)}) \quad \frac{\partial g(\mathbf{x})}{\partial \alpha_2} = \frac{g_2}{\alpha_2}(\mathbf{x}^{(2)})$$

# Residual Network

- Deep Neural Network:  $f_h(\mathbf{x}^{(h)}) := f_h(\mathbf{x}^{(h)}, \theta_h)$

$$f(\mathbf{x}) = f_H \circ f_{H-1} \circ \dots \circ f_0(\mathbf{x})$$

- Let  $f_h = g_2 \circ g_1 \circ g_0$  with *residual connection*

$$\mathbf{x} \xrightarrow{g_0} \oplus \rightarrow \mathbf{x}^{(1)} \xrightarrow{g_1} \oplus \rightarrow \mathbf{x}^{(2)} \xrightarrow{g_2} \oplus \rightarrow f_h(\mathbf{x})$$

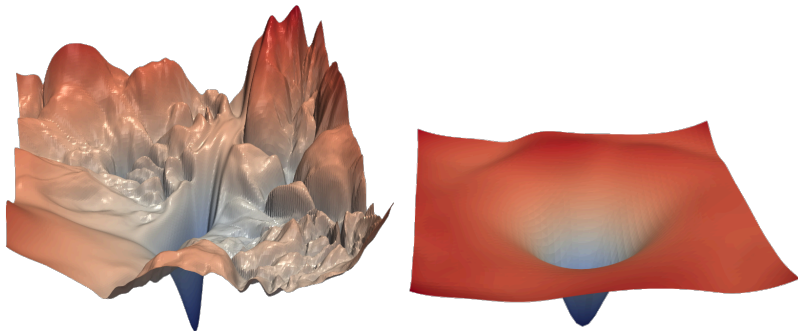

$$\frac{\partial f_h(\mathbf{x})}{\partial \alpha_0} = \frac{g_2}{g_1} \circ \frac{g_1}{g_0} \circ \frac{g_0}{\alpha_0}(\mathbf{x}) + \frac{g_1}{g_0} \circ \frac{g_0}{\alpha_0}(\mathbf{x}) + \frac{g_0}{\alpha_0}(\mathbf{x})$$

- ★ Improve gradient propagation
- ★ Prevent vanishing/shattered gradient

# Residual Network

- Deep Neural Network:  $f_h(\mathbf{x}^{(h)}) := f_h(\mathbf{x}^{(h)}, \theta_h)$

$$f(\mathbf{x}) = f_H \circ f_{H-1} \circ \dots \circ f_0(\mathbf{x})$$



From Hao Li et al. “Visualizing the Loss Landscape of Neural Nets”. In: *Proceedings of the 32nd International Conference on Neural Information Processing Systems*. NIPS’18. Montréal, Canada: Curran Associates Inc., 2018, pp. 6391–6401

This work is licensed under a [Creative Commons “Attribution-ShareAlike 4.0 International”](#) license.

